

STUDY NOTES

Java Object-Oriented Programming

Concepts, Syntax & Worked Examples — Chapters 1–7

OOP Fundamentals · Class & Object · Constructors · Encapsulation
Access Specifiers · Polymorphism · Abstraction

Java OOP — Complete Notes with Examples

1. OOP Main Concepts and Java Features

Main Concepts of OOP

Object-Oriented Programming is built on four pillars:

1. **Encapsulation** – Wrapping data (variables) and code (methods) together as a single unit (class), and restricting direct access to data using access modifiers.
2. **Abstraction** – Hiding internal implementation details and showing only the essential features to the user.
3. **Inheritance** – A mechanism where one class acquires the properties and behaviors of another class.
4. **Polymorphism** – The ability of an object to take many forms; the same method behaves differently based on the object calling it.

Other supporting concepts:

- **Class** – Blueprint/template for creating objects.
- **Object** – An instance of a class.
- **Message Passing** – Objects communicate by calling each other's methods.

Features of Java

- **Simple** – Syntax is similar to C/C++ but removes complex features like pointers, operator overloading, and multiple inheritance (via classes).
- **Object-Oriented** – Almost everything in Java is an object.
- **Platform-Independent** – (explained below)
- **Secure** – No explicit pointers; runs inside a JVM sandbox; classloader and bytecode verifier add security.
- **Robust** – Strong memory management, automatic garbage collection, exception handling.
- **Multithreaded** – Can perform many tasks simultaneously.
- **Architecture-Neutral** – No implementation-dependent data type sizes.
- **Portable** – Bytecode can run on any device with a JVM.

- **High Performance** – Uses Just-In-Time (JIT) compiler.
- **Distributed** – Supports networking (RMI, EJB) for building distributed applications.
- **Dynamic** – Supports dynamic loading of classes at runtime.

Platform-Independent Language

Java is called platform-independent because Java source code is compiled into an intermediate form called **bytecode** (`.class` file), not directly into machine code. This bytecode is not tied to any specific operating system or hardware. Any machine that has a **JVM (Java Virtual Machine)** installed can execute this bytecode, regardless of the underlying OS (Windows, Linux, Mac).

Example:

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```

Compiling `Hello.java` with `javac` produces `Hello.class` (bytecode). This same `Hello.class` file can run unmodified on Windows, Linux, or Mac — as long as each has a JVM.

“Write Once, Run Anywhere” (WORA) Principle

This is Java’s core slogan, meaning a program written and compiled once can run on any platform without recompilation, because:

- The compiler (`javac`) converts `.java` source code into platform-independent **bytecode**.
- Each platform has its own JVM implementation that translates this bytecode into native machine instructions for that specific platform.

So the *code* doesn’t change — only the JVM underneath changes per platform.

Five Phases of a Typical Java Program Execution

1. **Edit** – The programmer writes the source code in a text editor/IDE and saves it as a `.java` file (e.g., `Hello.java`).
2. **Compile** – The `javac` compiler translates the source code into bytecode, producing a `.class` file (e.g., `Hello.class`). Compilation errors (syntax errors) are caught here.
3. **Load** – The **Class Loader** (part of JVM) loads the `.class` file(s) into memory.

4. **Verify** – The **Bytecode Verifier** checks that the bytecode does not violate Java's security restrictions (e.g., no illegal memory access).
5. **Execute** – The JVM's execution engine (interpreter + JIT compiler) executes the bytecode, translating it into native machine code that the CPU can run.

Source Code (.java) → Compiler (javac) → Bytecode (.class) → Class Loader → Bytecode Verifier → JVM Execution Engine → Output

Attribute of an Object

An **attribute** (also called a field or instance variable) represents the **state** or **characteristics/properties** of an object. Each object has its own copy of the attribute values.

Example:

```
class Car {
    String color;    // attribute
    String model;    // attribute
    int speed;       // attribute
}

public class Main {
    public static void main(String[] args) {
        Car c1 = new Car();
        c1.color = "Red";
        c1.model = "Tesla";
        c1.speed = 120;
    }
}
```

Here, `color`, `model`, and `speed` are attributes describing the state of the `Car` object.

Java Virtual Machine (JVM)

The JVM is an abstract computing machine that provides a runtime environment in which Java bytecode can be executed. It is responsible for:

- Loading code (Class Loader Subsystem)
- Verifying code (Bytecode Verifier)
- Executing code (Execution Engine)
- Providing runtime memory areas (Heap, Stack, Method Area)

The JVM is **platform-dependent** (a different JVM exists for Windows, Linux, Mac), but the bytecode it runs is **platform-independent** — this is what makes Java portable.

Concept of Bytecode and JVM

- **Bytecode** is the intermediate, platform-independent code generated by the Java compiler (`javac`) from `.java` source files. It is stored in `.class` files and consists of a set of instructions that the JVM understands (not directly understood by the OS/CPU).
- **JVM** takes this bytecode and either interprets it line-by-line or uses a **JIT (Just-In-Time) compiler** to convert frequently-used bytecode into native machine code for faster execution.

Flow:

```
MyProgram.java --(javac)--> MyProgram.class (bytecode) --(JVM)--> Machine  
Code (platform-specific)
```

2. Class, Method, Object

Class

A **class** is a user-defined blueprint or template from which objects are created. It defines attributes (variables) and behaviors (methods) that its objects will have. A class itself does not occupy memory for data until an object is created.

Method

A **method** is a block of code that performs a specific task/behavior. It is defined inside a class and can operate on the object's attributes.

Object

An **object** is a real-world entity and an instance of a class. It has a state (attributes) and behavior (methods). Objects are created using the `new` keyword.

Box Example (Classic Example)

```
class Box {
    // attributes (state)
    double length;
    double breadth;
    double height;

    // method (behavior) to calculate volume
    double getVolume() {
        return length * breadth * height;
    }

    // method to display dimensions
    void display() {
        System.out.println("L: " + length + ", B: " + breadth + ", H: " +
height);
    }
}

public class Main {
    public static void main(String[] args) {
        // creating an object of Box
        Box box1 = new Box();

        // assigning values to attributes
        box1.length = 10;
        box1.breadth = 5;
        box1.height = 2;

        box1.display();
        System.out.println("Volume of box1: " + box1.getVolume());

        // another object with different state
        Box box2 = new Box();
        box2.length = 3;
        box2.breadth = 3;
        box2.height = 3;
        System.out.println("Volume of box2: " + box2.getVolume());
    }
}
```

Output:

```
L: 10.0, B: 5.0, H: 2.0
Volume of box1: 100.0
Volume of box2: 27.0
```

This shows `Box` as the class (blueprint), `length`, `breadth`, `height` as attributes, `getVolume()` and `display()` as methods, and `box1`, `box2` as two independent objects with their own state.

3. Constructor

Constructor and Its Types with Example

A **constructor** is a special method used to initialize objects. It has the same name as the class and no return type (not even `void`). It is automatically called when an object is created with `new`.

Types of Constructors:

a) Default Constructor — automatically provided by Java if no constructor is written; takes no arguments and initializes fields with default values.

```
class Student {
    String name;
    int age;
    // no constructor written -> Java provides a default one implicitly
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(); // default constructor called
        System.out.println(s1.name + " " + s1.age); // null 0
    }
}
```

b) No-Argument Constructor (explicitly defined) — a constructor written by the programmer that takes no parameters.

```

class Student {
    String name;
    int age;

    Student() { // no-arg constructor
        name = "Unknown";
        age = 18;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        System.out.println(s1.name + " " + s1.age); // Unknown 18
    }
}

```

c) Parameterized Constructor — accepts arguments to initialize an object with specific values at the time of creation.

```

class Student {
    String name;
    int age;

    Student(String n, int a) { // parameterized constructor
        name = n;
        age = a;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Rafi", 20);
        Student s2 = new Student("Nadia", 22);
        System.out.println(s1.name + " " + s1.age); // Rafi 20
        System.out.println(s2.name + " " + s2.age); // Nadia 22
    }
}

```

d) Copy Constructor — (Java doesn't have a built-in one like C++, but you can create one manually) used to create a new object as a copy of an existing object.


```

class Student {
    String name;
    int age;

    Student(String n, int a) {
        name = n;
        age = a;
    }

    // copy constructor
    Student(Student s) {
        name = s.name;
        age = s.age;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Rafi", 20);
        Student s2 = new Student(s1); // copy of s1
        System.out.println(s2.name + " " + s2.age); // Rafi 20
    }
}

```

e) Constructor Overloading — having multiple constructors in a class with different parameter lists.

```

class Student {
    String name;
    int age;

    Student() {
        name = "Unknown";
        age = 0;
    }

    Student(String n) {
        name = n;
        age = 18;
    }

    Student(String n, int a) {
        name = n;
        age = a;
    }
}

```

“this” Keyword in Java

this is a reference variable that refers to the **current object** — the object on which the method or constructor is being invoked. Common uses:

1. **To resolve naming conflict** between instance variables and parameters:

```

class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name; // this.name = instance variable, name = parameter
        this.age = age;
    }
}

```

1. **To call one constructor from another (constructor chaining)** within the same class:

```

class Student {
    String name;
    int age;

    Student() {
        this("Unknown", 18); // calls the parameterized constructor
    }

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

```

1. To pass the current object as a **parameter** to another method.
2. To return the current object from a method (useful in method chaining).

4. Data Hiding and Encapsulation

Data Hiding in Java

Data hiding is the technique of restricting direct access to an object's internal data (fields) from outside the class, typically by declaring them `private`. This protects data from unauthorized or accidental modification and enforces controlled access through public methods.

```

class BankAccount {
    private double balance; // hidden from outside classes

    public BankAccount(double balance) {
        this.balance = balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(1000);
        // acc.balance = -5000; // NOT ALLOWED - compile error, balance is
private
    }
}

```

Encapsulation

Encapsulation is the OOP principle of binding data (variables) and the methods that operate on that data into a single unit (class), while restricting direct access to some components. It is achieved by:

1. Declaring instance variables as `private`.
2. Providing `public` getter and setter methods to access and update the values.

Encapsulation is often called a “protective wrapper” around the data. A class that fully implements this is called an **encapsulated class**.

Getter and Setter Methods with Use

- **Getter** – A method used to *read/retrieve* the value of a private variable (usually named `getXxx()`).
- **Setter** – A method used to *modify/update* the value of a private variable (usually named `setXxx()`), often with validation logic.

Example (full encapsulation):

```

class BankAccount {
    private double balance; // private data - hidden

    // Constructor
    public BankAccount(double balance) {
        this.balance = balance;
    }

    // Getter
    public double getBalance() {
        return balance;
    }

    // Setter with validation
    public void setBalance(double balance) {
        if (balance >= 0) { // prevents invalid data
            this.balance = balance;
        } else {
            System.out.println("Balance cannot be negative!");
        }
    }

    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(5000);

        System.out.println("Initial Balance: " + acc.getBalance()); // using
getter

        acc.deposit(1500);
        System.out.println("After Deposit: " + acc.getBalance());

        acc.setBalance(-2000); // invalid, rejected by setter validation
        System.out.println("Balance after invalid set: " + acc.getBalance());
    }
}

```

Output:

```
Initial Balance: 5000.0
After Deposit: 6500.0
Balance cannot be negative!
Balance after invalid set: 6500.0
```

This demonstrates why getters/setters are useful: they let the class control **how** data is read or changed (e.g., validation), instead of allowing free, unchecked direct access.

5. Access Specifier in Java

Access specifiers (modifiers) control the visibility/accessibility of classes, methods, and variables from other classes.

Modifier	Same Class	Same Package	Subclass (different package)	Other Packages
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✗
<code>default</code> (no modifier)	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗

1. public

Accessible from **anywhere** — any class, in any package.

```
public class Employee {
    public String name; // accessible everywhere
}
```

2. private

Accessible **only within the same class**. Most restrictive; used for data hiding.

```
class Employee {
    private double salary; // accessible only inside Employee class

    private void showSalary() {
        System.out.println(salary);
    }
}
```

3. protected

Accessible within the **same package**, and also in **subclasses** even if they are in a different package (via inheritance).

```
class Employee {
    protected String department;
}

class Manager extends Employee {
    void display() {
        System.out.println(department); // accessible - Manager is a subclass
    }
}
```

4. default (no keyword specified)

Accessible only within the **same package**. If you don't write any access modifier, Java applies default access.

```
class Employee {
    int employeeId; // no modifier = default access, accessible only within same package
}
```

Combined Example:

```
class Employee {
    public String name;           // accessible everywhere
    protected String department; // accessible in package + subclasses
    int employeeId;              // default - accessible only in same package
    private double salary;       // accessible only inside this class

    private double getSalary() {
        return salary;
    }
}
```

6. Polymorphism

Polymorphism ("many forms") allows the same method name or reference to behave differently depending on the context (object or arguments). Java supports two types:

- **Compile-time (static) polymorphism** → Method Overloading
- **Runtime (dynamic) polymorphism** → Method Overriding

Super Keyword

`super` is a reference variable used to refer to the **immediate parent class object**. Uses:

1. To call the parent class's constructor.
2. To access the parent class's methods (especially when overridden).
3. To access the parent class's variables (when hidden by subclass variables with same name).


```

class Animal {
    String sound = "Some generic sound";

    Animal() {
        System.out.println("Animal constructor called");
    }

    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    String sound = "Bark";

    Dog() {
        super(); // calls Animal's constructor
        System.out.println("Dog constructor called");
    }

    void makeSound() {
        super.makeSound(); // calls Animal's version first
        System.out.println("Dog barks: " + super.sound + " -> " + this.sound);
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.makeSound();
    }
}

```

Output:

```

Animal constructor called
Dog constructor called
Animal makes a sound
Dog barks: Some generic sound -> Bark

```

Method Overloading (Compile-Time Polymorphism)

Multiple methods in the **same class** with the **same name** but **different parameter lists** (different number, type, or order of parameters). Decided at **compile time**.

```

class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(2, 3));           // 5 -> int version
        System.out.println(calc.add(2.5, 3.5));       // 6.0 -> double version
        System.out.println(calc.add(1, 2, 3));        // 6 -> three-parameter
    }
}

```

Method Overriding (Runtime Polymorphism)

A **subclass** provides its own specific implementation of a method that is already defined in its **parent class**, with the **same name, same return type, and same parameters**. Decided at **runtime**, based on the actual object type (not the reference type).

```

class Shape {
    void area() {
        System.out.println("Area of shape is undefined");
    }
}

class Circle extends Shape {
    double radius = 5;

    @Override
    void area() {
        System.out.println("Area of circle: " + (3.1416 * radius * radius));
    }
}

class Rectangle extends Shape {
    double length = 4, width = 6;

    @Override
    void area() {
        System.out.println("Area of rectangle: " + (length * width));
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s;                // parent class reference

        s = new Circle();        // runtime polymorphism
        s.area();                // calls Circle's area() at runtime

        s = new Rectangle();
        s.area();                // calls Rectangle's area() at runtime
    }
}

```

Output:

```

Area of circle: 78.54
Area of rectangle: 24.0

```

Here, the same reference type `Shape` calls a *different* `area()` implementation depending on the actual object (`Circle` or `Rectangle`) assigned to it at runtime — this is dynamic/runtime polymorphism.

7. Abstraction

Abstraction is the process of hiding the internal implementation details and showing only the essential/relevant features of an object to the user. It focuses on **what** an object does rather than **how** it does it. In Java, abstraction is achieved using:

1. **Abstract classes** (partial abstraction, 0–100%)
2. **Interfaces** (full abstraction, especially before Java 8)

Abstraction with Example

Using Abstract Class:

```

abstract class Vehicle {
    // abstract method - no body, must be implemented by subclasses
    abstract void start();

    // concrete method - has implementation, shared by all subclasses
    void fuelType() {
        System.out.println("This vehicle runs on some type of fuel.");
    }
}

class Car extends Vehicle {
    @Override
    void start() {
        System.out.println("Car starts with a key/button ignition.");
    }
}

class Bike extends Vehicle {
    @Override
    void start() {
        System.out.println("Bike starts with a kick or self-start.");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v1 = new Car();    // cannot do: new Vehicle() - abstract class
        // can't be instantiated
        Vehicle v2 = new Bike();

        v1.start();                // Car starts with a key/button ignition.
        v1.fuelType();             // This vehicle runs on some type of fuel.

        v2.start();                // Bike starts with a kick or self-start.
    }
}

```

Using Interface (100% abstraction):

```

interface Payable {
    void pay(double amount); // implicitly public and abstract
}

class CreditCardPayment implements Payable {
    @Override
    public void pay(double amount) {
        System.out.println("Paid $" + amount + " using Credit Card.");
    }
}

class UpiPayment implements Payable {
    @Override
    public void pay(double amount) {
        System.out.println("Paid $" + amount + " using UPI.");
    }
}

public class Main {
    public static void main(String[] args) {
        Payable p1 = new CreditCardPayment();
        Payable p2 = new UpiPayment();

        p1.pay(500); // Paid $500.0 using Credit Card.
        p2.pay(250); // Paid $250.0 using UPI.
    }
}

```

Key idea: The user of `Vehicle` or `Payable` only needs to call `start()` or `pay()` — they don't need to know *how* the engine ignites or *how* the payment gateway processes the transaction internally. That complexity is hidden — this is abstraction.

Quick Summary Table

Concept	Achieved By
Encapsulation	private fields + public getters/setters
Abstraction	abstract classes / interfaces
Inheritance	<code>extends</code> / <code>implements</code>
Polymorphism	Method overloading (compile-time), Method overriding (runtime)